

Your model is not brilliant. It may be cheating.

A practical data leakage cheat sheet
for the AI era.

What leakage is

Target leakage

Time leakage

Train/test contamination

Choosing a split

Red flags

Pipelines in scikit-learn

A review checklist

Sagar Rathkanthiwar

Data & AI Professional

10 slides · save for later →

Information from the wrong side of the line

Data leakage is when a model trains on — or is scored with — information it would not actually have at prediction time. The score looks great because the exam contained the answers.



← leakage is any value that crosses this line backwards

- **Ask one question of every column:** at the exact moment I need a prediction, does this value exist yet?
- **If the answer is “no” or “only afterwards,”** it is leakage — no matter how predictive it looks.

ONE EXAMPLE — PREDICTING WHO WILL CANCEL NEXT MONTH

tickets_opened_last_30d

already true today — safe to use

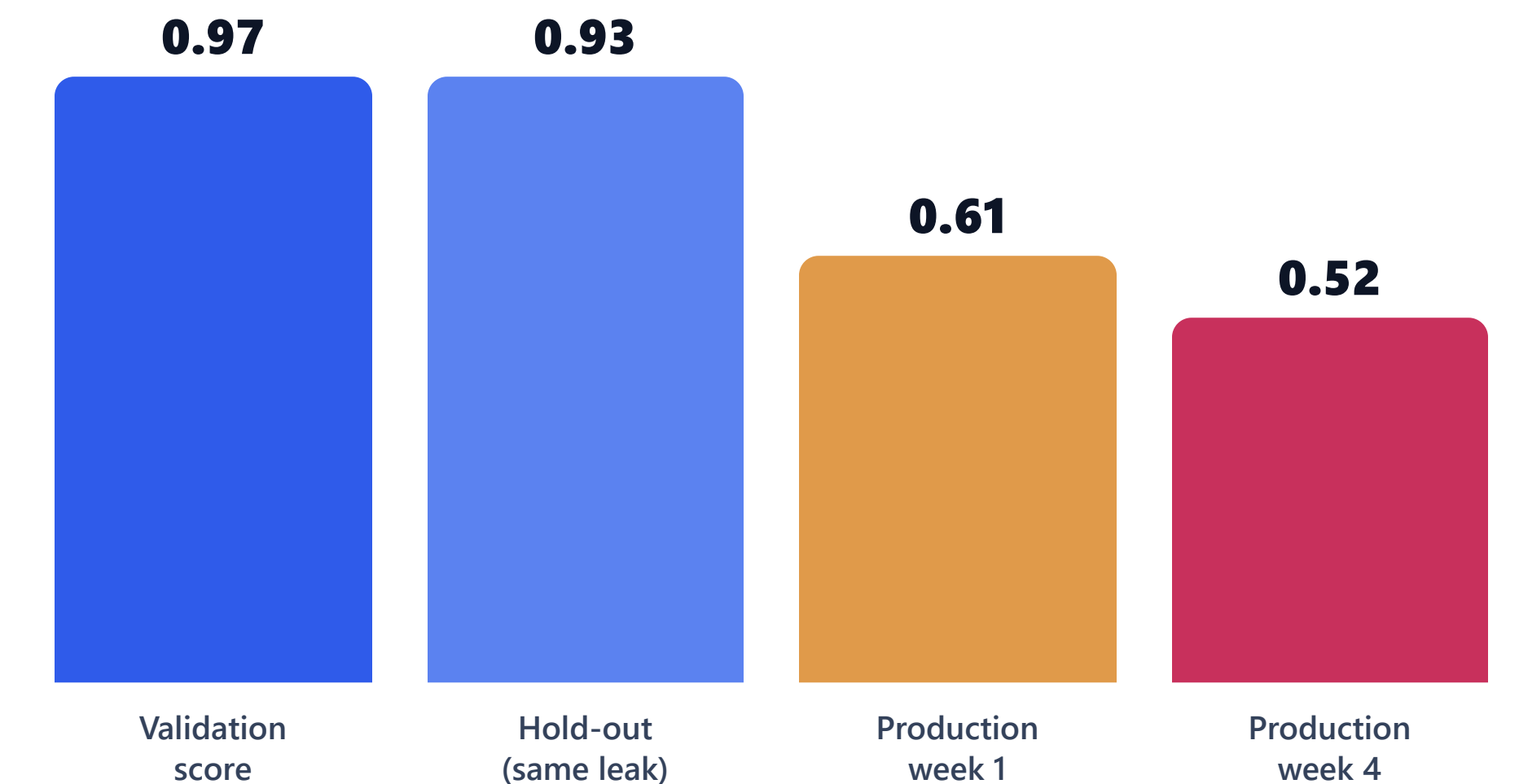
cancellation_survey_score

only filled in *after* they cancel — leakage

Leakage is not a flaw in the algorithm. It is a **mistake in how the data and the evaluation were set up** — which is exactly the part no model can check for you.

It fails *after* you ship it

Leakage never announces itself. It shows up as a number so good that nobody questions it — until the model meets real data.



Illustrative pattern, not measured data — the **shape** is the point

Notice the trap: **the hold-out set agreed**. If the leak is in your data, every split you carve out of that data carries it too — so the second opinion was never independent. **Production is the first honest test.**

The feature that quietly is the answer

Some columns are created **because** of the outcome, or at the same moment as it. Include them and the model simply reads the label back.

THE TASK

Predict whether a **loan application will be approved** — at the moment it is submitted.



COLUMNS IN THE TRAINING TABLE

interest_rate_offered

only set once approved

funds_disbursed_on

happens after the decision

assigned_loan_officer

only assigned to approvals

rejection_reason_code

literally the label

credit_score · income · debt_ratio

known at submission — keep

Four of these five columns do not exist yet at the moment you need the prediction.

The indirect leaks are the dangerous ones. Nobody ships `rejection_reason_code`. But a blank `assigned_loan_officer` means “rejected” just as reliably — and looks like an innocent feature in the schema.

Letting the future teach the past

If your data has a timeline — churn, fraud, demand, prices, clicks — a **random** split scatters future rows into training. The model learns from events that had not happened yet.

RANDOM SPLIT — WRONG HERE



JAN → DEC

Test rows sit **before** training rows. To score January, the model has already read December.

TIME-BASED SPLIT — CORRECT



JAN → DEC

Train on the past, test on the future — **the same order production will face.**

■ training rows ■ test rows · each block is one month

```
# WRONG for time-ordered data - shuffling scrambles the calendar
train_test_split(X, y, test_size=0.2, shuffle=True)

# RIGHT - expanding windows that always score a LATER slice
for train_idx, test_idx in TimeSeriesSplit(n_splits=5).split(X):
    ...           # every fold trains on the past, tests on the next period
```

This applies **inside** a feature too. A “customer lifetime value” or “total complaints” column computed over the whole dataset already contains the months you are trying to predict.

Fit on train only. Every time.

Scaling, imputing, encoding, feature selection — each one **learns statistics from the data you hand it**. Fit on everything and the test set has already shaped the model.

```
# WRONG - the scaler saw the test rows' mean and standard deviation
X_scaled = StandardScaler().fit_transform(X)           # all rows
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y)

# RIGHT - a Pipeline refits every step inside each fold
# sklearn: Pipeline, SimpleImputer, StandardScaler,
#           LogisticRegression, train_test_split, cross_val_score
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")),        # median from TRAIN only
    ("scale", StandardScaler()),                        # mean/std from TRAIN only
    ("model", LogisticRegression(max_iter=1000)),
])

scores = cross_val_score(pipe, X_train, y_train, cv=5, scoring="roc_auc")
pipe.fit(X_train, y_train)                             # test set still untouched
```

A **Pipeline** is not a style preference — it is the leak guard. It makes “fit on train only” true automatically inside every cross-validation fold, which is exactly where it is easiest to get wrong by hand.

The split *is* the experiment

Your split should imitate the moment of deployment. Pick it from the structure of the data — not from habit.

Random split

WHEN rows are independent and order does not matter — one image, one patient, one document per row.

```
train_test_split(X, y, stratify=y)
```

Time-based split

WHEN anything about the data is a timeline — churn, fraud, demand, prices, sessions.

```
TimeSeriesSplit(n_splits=5)
```

Group-based split

WHEN one entity produces many rows — a customer, a patient, a device, a store. Keep it wholly on one side.

```
GroupKFold(n_splits=5).split(X, y, groups=customer_id)
```

```
# Time-aware: sort first, then split forward - never shuffle
df = df.sort_values("event_date")
cutoff = "2026-07-01" # train on the past, test on the future
train = df[df["event_date"] < cutoff]
test = df[df["event_date"] >= cutoff]
```

Real data often has **two** of these structures at once — repeat customers *and* a timeline. The right choice always depends on your data and how the model is actually deployed.

Five signs not to celebrate yet

Leakage rarely arrives as an error message. It arrives as a result that is **a little too good**. These are the tells.

1. Suspiciously high score

0.99 AUC on a messy human problem is a bug report, not a result. Compare it against a trivial baseline.

2. One feature dominates

Drop your top feature. If the score collapses, go read that column's definition very carefully.

3. Post-outcome columns

Anything named `_final`, `_closed`, `_resolved`, `_total` — or carrying a timestamp after the decision point.

4. The same entity on both sides

One customer with 40 rows split across train and test means the model memorised them, not the pattern.

5. Production performance falls off a cliff

A model that scores well offline and poorly in week one is the classic signature. Before you retrain or swap algorithms, re-audit the features, the timeline and the split — that is usually where the answer is hiding.

The fastest sanity check there is: **can you tell a colleague a plausible story for *why* each top feature predicts the outcome?** If the only story is “because it already knows,” you have found your leak.

AI can build the pipeline. You own the validity.

A model will happily generate a clean, well-structured, **completely invalid** pipeline — because leakage lives in facts about your business that were never in the code.

- 1 Would this feature exist at prediction time?** Column by column. Not “is it in the table” — *is it populated yet*.
- 2 Was preprocessing fit on training data only?** Every scaler, imputer, encoder and feature selector, inside every fold.
- 3 Does the split reflect real deployment?** If production predicts forward in time, the evaluation must too.
- 4 Could the same customer or entity land on both sides?** Check duplicates and repeated IDs before you trust any score.
- 5 What is the exact prediction timestamp?** Write it down. Every question above is answered relative to that one moment.

A PROMPT WORTH REUSING

Review this ML pipeline for **data leakage**. For each feature, state whether it would be available at prediction time. Flag any preprocessing fit outside the training fold, and say whether the split matches this deployment. **List the risks before suggesting any fix.**

A model that sees the future is not intelligent. It is invalid.

- Leakage is **information from the wrong side of the prediction moment** — in a feature, in the timeline, or in the split.
- Wrap every transformation in a **Pipeline** so “fit on train only” stops depending on memory.
- Choose the split from the **structure of the data**: random, time-based, or grouped by entity.
- AI can generate the whole pipeline in seconds. **Validating the data, the timeline and the evaluation design stays human work.**
- When a score looks too good, **treat it as a bug report** — not a result to announce.

**What data-quality or evaluation issue has surprised you most?
Share it below — someone is about to ship that exact bug.**

Sagar Rathkanthiwar · Data & AI Professional · follow for more field guides

Note: examples target scikit-learn 1.x. The right split and the right feature set always depend on your data and how the model is deployed — treat this as a review checklist, not a recipe.